Time-Travel Debugger

Project Overview

The Time-Travel Debugger (TTDB) is built on a client-server architecture where the client sends a program trace and the server generates a `.tdbg` file in return. This `.tdbg` file allows the client to navigate the program's execution history backward and forward, observing the program state at any specific line, essentially enabling "time travel" debugging.

Phases:

The project has two main phases. **In phase 01**, you have to implement the server-side, which takes the source.bin. Then it works on this and returns a .tdbg file to the client. In **phase 02**, you will implement the client-side. The **deadline** for phase 01 is **5th October, 2026.**

PHASE 01

Pipeline Architecture:

Stage 0: Receive - We assume that we have a source.bin file from the client. We work on this file in this phase.
The code language would be our custom language C--; this is a very simple language and has an **Instruction set {func, func_end, call, set, add, mul, sub, div}**. So, the code file the server receives is built using this language.
Each line of code is built using the rule → **Keyword Identifier Params/Args**
The **maximum number of params** is **16**.

Demo Code:

```
func foo b
set a 10
add b a
func_end
func main
set k 10
```

call foo k
func_end
In the add, sub, mul, div instructions, the result is saved in the first parameter; in this case, it is *b*. And the updated value of b will automatically reflect in the main function.

Stage 1: Pass 0x0: Validation - Scans raw instructions to enforce structural integrity.

Verifies 1:1 FUNC / FUNC_END matching and explicitly rejects nested declarations.
If the structural integrity fails, it sends the error to the client.
For this implementation, figure out which data structure is most suitable.
Valid Code:
func foo a b
// something done
func_ end
Invalid Code:

1. func foo a
   // something done
2. func foo n
   // something done
   func fee n
   func_end
   func_end

Stage 2: Pass 0x1: Resolve - Formats source lines into structured records with line byte offsets in resolve.bin. Tracks symbol destinations and patches target addresses for CALL instructions.

Each line of code should be stored in resolve.bin in this format →
[offset(8B)][string_size(4B)][string]
Before each string of code, there should be an offset and the size of the string. The next offset would be calculated using this formula → **offset = offset + 8 + 4 + string_size**. The first offset would be 0x0.
While creating the resolve.bin, you also need to resolve the call instruction such that if **func foo** is at **0x20** and another instruction *call foo b* is in the code, we will resolve it like *call 0x20 b*. For this, during the creation of the bin file, you have to keep a record of the function names and their offsets, and the call instruction that called xyz func and the offset of this instruction. After the creation of this file, you will replace the func_name in the call instruction with the correct offset.

These two structs are given for bookkeeping:

```
struct FuncEntry
{
  string funcName;
  int64_t byteOffsetInResolveBin; // where this function's FUNC header record sits
};
struct PendingPatch
{
  int64_t byteOffsetOfOffsetField; // where in resolve.bin to set back and overwrite
  string targetFuncName;
};
```

Resolve returns the offset of the main function. If main doesnt exist, we do not execute this file and send the client the error, also in the case of a call to an undefined function.

Stage 03: Pass 0x2: Execution - Tokenizes and executes instructions step-by-step.

Manages a call stack and creates snapshot captures at every tick.
During execution, you take one line of code, tokenize it, and execute the instruction.
There is a **callStack** that contains a frame for each function call. At the start of the execution function, we first create the frame on the callStack for main. From that onwards, at each call instruction, the frame would be created on the callStack for the function called.
The execution will start from the main function. Along with the stack, we also have to maintain the timeline. The timeline is a doubly linked list of the snapshots. After execution of each line we create a snapshot.

```
enum TokenType
{
  KEYWORD,
  IDENTIFIER,
  PARAM
};
struct Token
{
  TokenType type;
  string text;
};

struct Variable
```

```
{
  string name;
  int32_t value;
};
struct Frame
{
  string func_name;
  int32_t argc;
  Variable argv[MAX_VARS_PER_FRAME];
  int32_t returnLine;
  Variable locals[MAX_VARS_PER_FRAME];
  int32_t localCount;
};
struct Snapshot
{
  Frame callStack[MAX_STACK_DEPTH];
  int32_t stackDepth;
};
struct TimelineNode
{
  Snapshot *data;
  TimelineNode *next;
  TimelineNode *prev;
};
class Timeline
{
  TimelineNode *head, *tail;
  int32_t stepCount;

public:
  // Implement these functions
  Timeline()
  {
  }
  void record(Snapshot *s)
  {
```

```
    // add record in the timeline
  }
  TimelineNode *begin()
  {
  }
  int32_t getStepCount()
  {
  }
};
```

Stage 04: Pass 0x3: Serialization - Serializes the complete timeline to disk, formatting the custom .tdbg structure containing the fixed header, snapshot records, and dense index.

The header contains magic bytes 'TTDB', version, stepCount, and indexOffset. While pushing the timeline node in the disk, you have to maintain the index array. At the end the index array will also be added to the file. It helps in the search of the nth snapshot of the timeline.

```
struct TTDBHeader
{
  char magic[4]; // "TTDB"
  int32_t version;
  int32_t stepCount;
  int64_t indexOffset;
};
void writeHeader(FILE *f, const TTDBHeader &h)
{
  fwrite(h.magic, 1, 4, f);
  fwrite(&h.version, sizeof(int32_t), 1, f);

  // placeholder for other two data members
}
void writeTdbg(Timeline &timeline, const char *tdbgPath)
{
  // placeholder for header
  // index array of the size of stepcount from the timeline
```

// placing each snapshot in the file while maintaining the index(starting point of each nth snapshot)

// after timeline add the index array i the file

// update the header

}

Output File Format (.tdbg)

The session.tdbg file layout is strictly partitioned into three segments:

- 1. Header Segment (TTDBHeader): Contains magic bytes 'TTDB', version, stepCount, and indexOffset.
- 2. Snapshot Stream: Sequential write of the Snapshot structs representing the program's memory state across every instruction.
- 3. Dense Index Trailer: An array of offsets mapping each step to its Snapshot record for immediate temporal seeking.

Deadline and Submission:

- As stated above, the Phase 01 deadline is **5th October, 2026**.
- Submit your GitHub repo link.